МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине «Аппаратное обеспечение интеллектуальных систем» за 5 семестр
Тема: «АНАЛИЗ СЕТЕВОГО ТРАФИКА И ПРОТОКОЛОВ НА БАЗЕ WIRESHARK»

Выполнил:
Студент ФЭИС
Группы ИИ-28
Каневега К.А.
(1 подгруппа)

Проверил:
Кулик А.Д.

Брест 2026

Цель работы: Изучение возможностей программы Wireshark, освоение типов фильтрации и приемов статистической обработки сетевого трафика. Практическое исследование структуры пакетов и принципов взаимодействия узлов на базе ключевых сетевых протоколов (IPv4, ICMP, TCP, UDP, ARP).

Ход работы:

ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

1. Изучить краткие теоретические сведения по возможностям, приемам работы с программой Wireshark (файл netWS.pdf).
2. Изучить: типы фильтрации трафика, правила построения фильтров, приемы статистической обработки сетевого трафика в Wireshark (материал приведен ниже).
3. Запустив Wireshark на захват, выполнить загрузку доступной в лабораторных условиях страницы (bstu.by, iit.bstu.by или др.). Остановить и сохранить захват. Для захваченных пакетов определить статистические данные:
   - процентное соотношение трафика разных протоколов в сети;
   - среднюю скорость кадров/сек;
   - среднюю скорость байт/сек;
   - минимальный, максимальный и средний размеры пакета;
   - степень использования полосы пропускания канала (загрузку сети).
4. Отфильтровать в захвате IP пакеты. Определить статистические данные:
   - процентное соотношение трафика разных протоколов стека tcp/ip в сети;
   - средний, минимальный, максимальный размеры пакета.

   На примере любого IP-пакета указать структуры протоколов Ethernet и IP. Отметить поля заголовков и описать их и интерпретировать их значения.

5. Запустив Wireshark на захват, выполнить команду ping для IP адреса соседней рабочей станции в лаборатории (предварительно определив ее адрес с помощью ipconfig). Сохранить результат. Сформировав нужный фильтр, отфильтровать пакеты, относящиеся к выполнению команды ping. На базе полученных пакетов и значений их полей интерпретировать результат работы утилиты ping. Описать все протоколы, используемые утилитой. Составить диаграмму взаимодействия машин при работе утилиты *ping*. Примечание. Данная утилита использует протокол ICMP (RFC 792 и RFC 960).

6. Запустив Wireshark на захват, выполнить разработанный в предыдущей лабораторной работе TCP-сервер и TCP-клиент. Сохранить результат. Сформировав нужный фильтр, отфильтровать пакеты данного сеанса. На примере любого TCP-сегмента указать структуру протокола TCP. Отметить поля заголовка, описать их и интерпретировать их значения. Составить диаграмму взаимодействия TCP-сервера и TCP-клиента.
7. Запустив Wireshark на захват, выполнить разработанный в предыдущей лабораторной работе UDP-сервер и UDP-клиент. Сохранить результат. Сформировав нужный фильтр, отфильтровать пакеты данного сеанса. На примере любого UDP-сегмента указать структуру протокола UDP. Отметить поля заголовка, описать их и интерпретировать их значения.
8. Выполнить анализ ARP-протокола по примеру из методических указаний.

Задание 3:

Wireshark · Статистика иерархии протоколов · Беспроводная сеть

Протокол	Процент пакетов	Пакеты	Процент байтов	Байты	Бит/с	Конечные пакеты	Конечные байты	Конечные бит/с	PDU
Frame	100.0	4282	100.0	3550961	693 k	0	0	0	4282
Ethernet	100.0	4282	1.7	61118	11 k	0	0	0	4282
Internet Protocol Version 6	0.6	25	0.0	1000	195	1	40	7	25
User Datagram Protocol	0.4	19	0.0	152	29	0	0	0	19
Multicast Domain Name System	0.3	11	0.0	1396	272	11	1396	272	11
Link-local Multicast Name Resolution	0.2	8	0.0	264	51	8	264	51	8
Internet Control Message Protocol v6	0.1	5	0.0	40	7	5	40	7	5
Internet Protocol Version 4	99.1	4244	2.4	84880	16 k	0	0	0	4244
User Datagram Protocol	50.4	2158	0.5	17264	3370	0	0	0	2158
Teredo IPv6 over UDP tunneling	0.1	6	0.0	345	67	0	0	0	6
Simple Service Discovery Protocol	0.4	17	0.1	2587	505	17	2587	505	17
QUIC IETF	47.9	2050	52.6	1869346	364 k	2050	1868492	364 k	2053
Multicast Domain Name System	0.3	11	0.0	1396	272	11	1396	272	11
Link-local Multicast Name Resolution	0.2	8	0.0	264	51	8	264	51	8
Domain Name System	0.8	34	0.1	3186	621	34	3186	621	34
Data	0.7	32	0.2	5526	1078	32	5526	1078	32
Transmission Control Protocol	48.7	2085	1.2	42828	8361	963	20352	3973	2085
Transport Layer Security	26.1	1119	46.0	1634272	319 k	1119	1300030	253 k	1400
Hypertext Transfer Protocol	0.0	2	0.0	319	62	1	154	30	2
Line-based text data	0.0	1	0.0	22	4	1	22	4	1
Data	0.0	1	0.0	1	0	1	1	0	1
Data	0.0	1	0.0	1480	288	1	1480	288	1
Address Resolution Protocol	0.4	19	0.0	532	103	19	532	103	19

Статистика

Измеряемые параметры	Захвачено	Показано	Отмечено
Пакеты	4282	4282 (100.0%)	—
Временной промежуток, с	40.979	40.979	—
В среднем, пакетов/с	104.5	104.5	—
Средний размер пакета, Б	829	829	—
Байты	3550961	3550961 (100.0%)	0
В среднем байт/с	86 k	86 k	—
В среднем бит/с	693 k	693 k	—

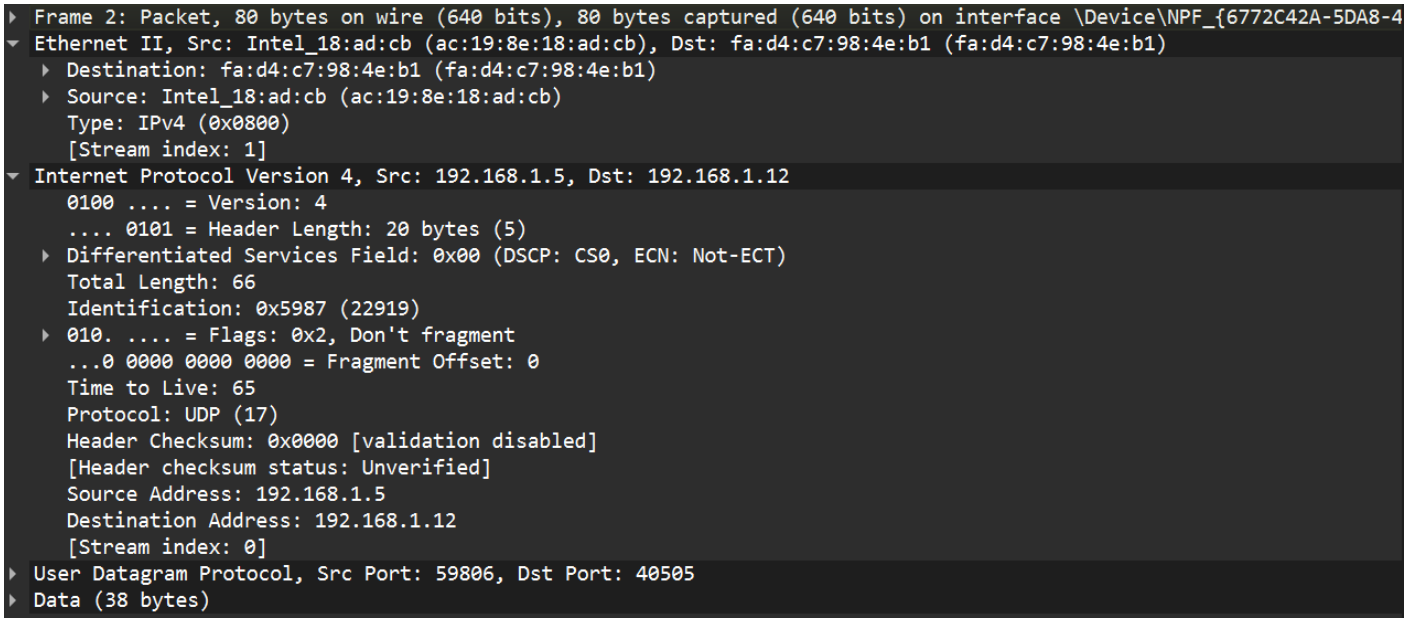
Wireshark · Packet Lengths · Беспроводная сеть

Topic / Item	Cou	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Start
Packet Lengths	4282	829,28	42	17478	0,1045	100%	3,9300	25,247
0-19	0	-	-	-	0,0000	0,00%	-	-
20-39	0	-	-	-	0,0000	0,00%	-	-
40-79	1324	60,85	42	79	0,0323	30,92%	0,4500	25,653
80-159	430	109,33	80	159	0,0105	10,04%	0,2400	25,943
160-319	198	210,70	161	318	0,0048	4,62%	0,1100	26,760
320-639	168	482,10	320	639	0,0041	3,92%	0,1600	25,650
640-1279	240	998,33	648	1276	0,0059	5,60%	0,1300	25,884
1280-2559	1759	1358,01	1284	2554	0,0429	41,08%	3,4300	25,247
2560-5119	124	3229,39	2568	4410	0,0030	2,90%	0,0500	38,287
5120 and greater	39	6971,59	5862	17478	0,0010	0,91%	0,0200	32,352

Задание 4:

Wireshark · Packet Lengths · Беспроводная сеть

Topic / Item	Cou	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Sta
Packet Lengths	4244	835,84	50	17478	0,1047	100%	3,9300	25,247
0-19	0	-	-	-	0,0000	0,00%	-	-
20-39	0	-	-	-	0,0000	0,00%	-	-
40-79	1305	61,12	50	79	0,0322	30,75%	0,4500	25,653
80-159	415	109,53	80	159	0,0102	9,78%	0,2400	25,943
160-319	198	210,70	161	318	0,0049	4,67%	0,1100	26,760
320-639	164	486,05	320	639	0,0040	3,86%	0,1600	25,650
640-1279	240	998,33	648	1276	0,0059	5,66%	0,1300	25,884
1280-2559	1759	1358,01	1284	2554	0,0434	41,45%	3,4300	25,247
2560-5119	124	3229,39	2568	4410	0,0031	2,92%	0,0500	38,287
5120 and greater	39	6971,59	5862	17478	0,0010	0,92%	0,0200	32,352



Структура протокола Ethernet II (Канальный уровень)

- **Destination (MAC-адрес получателя):** `fa:d4:c7:98:4e:b1`. Это физический адрес сетевого адаптера устройства, которому направлен кадр в локальной сети.
- **Source (MAC-адрес отправителя):** `Intel_18:ad:cb (ac:19:8e:18:ad:cb)`. Это физический адрес твоей сетевой карты (с чипом Intel), с которой был отправлен кадр.
- **Type (Тип протокола):** `IPv4 (0x0800)`. Это поле указывает, что внутрь кадра Ethernet вложен пакет сетевого уровня IPv4.

Структура протокола Internet Protocol Version 4 (Сетевой уровень)

- **Version (Версия):** `4`. Указывает на использование 4-й версии протокола IP.
- **Header Length (Длина заголовка):** `20 bytes (5)`. Размер заголовка IP-пакета. Значение 5 означает 5 слов по 32 бита, что в сумме дает стандартные 20 байт (без дополнительных опций).
- **Differentiated Services Field (Тип обслуживания):** `0x00`. Поле для приоритезации трафика. Значение 0x00 означает обычный трафик без специфического приоритета.
- **Total Length (Общая длина):** `66`. Общий размер пакета (20 байт IP-заголовка + 46 байт полезной нагрузки, включающей заголовок UDP и сами данные).
- **Identification (Идентификатор):** `0x5987 (22919)`. Уникальный номер пакета, который используется принимающей стороной для правильной сборки фрагментов, если пакет был разбит на части.
- **Flags (Флаги):** `0x2, Don't fragment`. Установлен бит, запрещающий промежуточным маршрутизаторам дробить этот пакет на более мелкие фрагменты.
- **Fragment Offset (Смещение фрагмента):** `0`. Поскольку пакет не фрагментирован, смещение равно нулю.
- **Time to Live (TTL, Время жизни):** `65`. Счетчик, определяющий, сколько маршрутизаторов (хопов) может пройти пакет до того, как будет уничтожен. С каждым переходом значение уменьшается на 1.
- **Protocol (Вложенный протокол):** `UDP (17)`. Указывает, что данные, переносимые в этом IP-пакете, принадлежат протоколу транспортного уровня UDP.
- **Header Checksum (Контрольная сумма):** `0x0000 [validation disabled]`. Контрольная сумма для проверки целостности заголовка. В данном случае проверка отключена на уровне ОС/драйвера для повышения производительности (offloading).
- **Source Address (IP-адрес отправителя):** `192.168.1.5`. Логический адрес машины, с которой отправлен пакет.
- **Destination Address (IP-адрес получателя):** `192.168.1.12`. Логический целевой адрес машины, которой адресован пакет.

Задание 5:

```
C:\AIPOS\lab3>ipconfig

Настройка протокола IP для Windows


Адаптер Ethernet Radmin VPN:

   DNS-суффикс подключения . . . . . :
   IPv6-адрес. . . . . . . . . . . . : fdfd::1aca:9dac
   Локальный IPv6-адрес канала . . . : fe80::abc0:9d7e:c058:ead4%8
   IPv4-адрес. . . . . . . . . . . . : 26.202.157.172
   Маска подсети . . . . . . . . . . : 255.0.0.0
   Основной шлюз. . . . . . . . . . : 26.0.0.1

Адаптер беспроводной локальной сети Подключение по локальной сети* 9:

   Состояние среды. . . . . . . . . : Среда передачи недоступна.
   DNS-суффикс подключения . . . . . :

Адаптер беспроводной локальной сети Подключение по локальной сети* 10:

   Состояние среды. . . . . . . . . : Среда передачи недоступна.
   DNS-суффикс подключения . . . . . :

Адаптер беспроводной локальной сети Беспроводная сеть:

   DNS-суффикс подключения . . . . . : bbrouter
   Локальный IPv6-адрес канала . . . : fe80::b089:9322:3bf7:7647%9
   IPv4-адрес. . . . . . . . . . . . : 192.168.1.5
   Маска подсети . . . . . . . . . . : 255.255.255.0
   Основной шлюз. . . . . . . . . . : 192.168.1.1

Адаптер Ethernet Сетевое подключение Bluetooth:

   Состояние среды. . . . . . . . . : Среда передачи недоступна.
   DNS-суффикс подключения . . . . . :
```

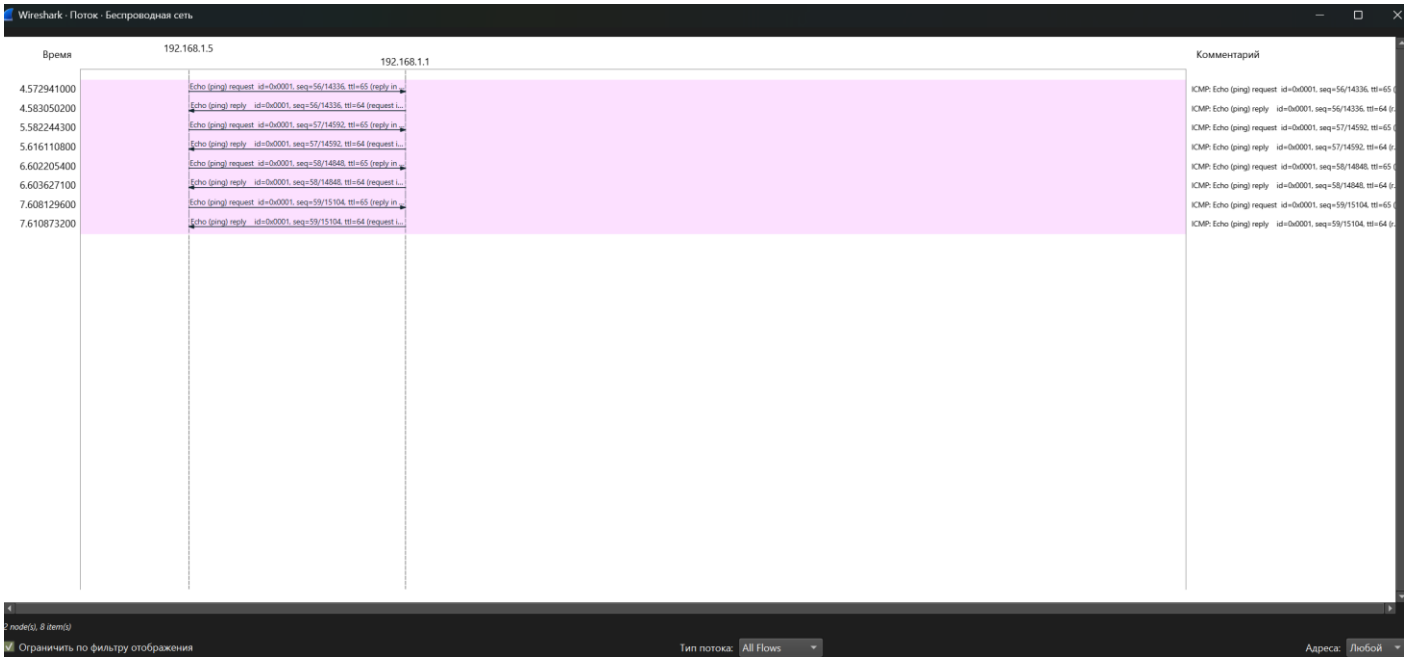
```
C:\AIPOS\lab3>ping 192.168.1.1

Обмен пакетами с 192.168.1.1 по с 32 байтами данных:
Ответ от 192.168.1.1: число байт=32 время=10мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=33мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=1мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=2мс TTL=64

Статистика Ping для 192.168.1.1:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 1мсек, Максимальное = 33 мсек, Среднее = 11 мсек
```

icmp

No.	Time	Source	Destination	Protocol	Length	Info
115	4.572941000	192.168.1.5	192.168.1.1	ICMP	74	Echo (ping) request id=0x0001, seq=56/14336, ttl=65 (reply in 116)
116	4.583050200	192.168.1.1	192.168.1.5	ICMP	78	Echo (ping) reply id=0x0001, seq=56/14336, ttl=64 (request in 115)
124	5.582244300	192.168.1.5	192.168.1.1	ICMP	74	Echo (ping) request id=0x0001, seq=57/14592, ttl=65 (reply in 125)
125	5.616110800	192.168.1.1	192.168.1.5	ICMP	78	Echo (ping) reply id=0x0001, seq=57/14592, ttl=64 (request in 124)
147	6.602205400	192.168.1.5	192.168.1.1	ICMP	74	Echo (ping) request id=0x0001, seq=58/14848, ttl=65 (reply in 148)
148	6.603627100	192.168.1.1	192.168.1.5	ICMP	78	Echo (ping) reply id=0x0001, seq=58/14848, ttl=64 (request in 147)
154	7.608129600	192.168.1.5	192.168.1.1	ICMP	74	Echo (ping) request id=0x0001, seq=59/15104, ttl=65 (reply in 155)
155	7.610873200	192.168.1.1	192.168.1.5	ICMP	78	Echo (ping) reply id=0x0001, seq=59/15104, ttl=64 (request in 154)



Задание 6:

```
[ПРОТОКОЛ] [2026-11-02 19:00:37] Начало соединения с 127.0.0.1:12345
Успешно подключено к серверу по умолчанию: 127.0.0.1:12345

Доступные команды:
 - create <имя_файла> <текст> (для отправки нажмите Home или Enter)
 - disconnect <адрес> <порт> (разрыв соединения)
 - exit (выход из программы)

> create test2 its test message
[ПРОТОКОЛ] [2026-11-02 19:01:03] Передана строка: "create test2 its test message" в 2026-11-02 19:01:03
Ответ сервера: FILE_SIZE: 16
> create test3 test
[ПРОТОКОЛ] [2026-11-02 19:09:15] Передана строка: "create test3 test" в 2026-11-02 19:09:15
Ответ сервера: FILE_SIZE: 4
>
```

tcp.port == 12345

No.	Time	Source	Destination	Protocol	Length	Info
35	32.429480600	127.0.0.1	127.0.0.1	TCP	63	49688 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=19
36	32.429598700	127.0.0.1	127.0.0.1	TCP	44	12345 → 49688 [ACK] Seq=1 Ack=20 Win=255 Len=0
37	32.431566700	127.0.0.1	127.0.0.1	TCP	58	12345 → 49688 [PSH, ACK] Seq=1 Ack=20 Win=255 Len=14
38	32.431593400	127.0.0.1	127.0.0.1	TCP	44	49688 → 12345 [ACK] Seq=20 Ack=15 Win=255 Len=0

- **Source Port (Порт отправителя):** `49688`. Это динамический порт, который операционная система выделила Java-клиенту.
- **Destination Port (Порт получателя):** `12345`. Это порт TCP-сервера, который задан в коде.
- **Sequence Number (Порядковый номер):** `Seq=1`. Номер первого байта данных в этом пакете относительно начала сессии. Используется для правильной сборки сообщения.
- **Acknowledgment Number (Номер подтверждения):** `Ack=1`. Указывает номер следующего байта, который клиент ожидает получить от сервера.
- **Flags (Флаги):** `[PSH, ACK]`.
    - `PSH` (Push) указывает принимающей стороне, что данные нужно немедленно передать приложению, не дожидаясь заполнения буфера.
    - `ACK` (Acknowledgment) означает, что поле номера подтверждения содержит валидное значение.
- **Window Size (Размер окна):** `Win=255`. Количество байт, которое отправитель готов принять (размер буфера).
- **Length (Длина данных):** `Len=19`. Размер полезной нагрузки (передаваемого текста) составляет 19 байт.

Интерпретация сеанса связи (диалога): На скриншоте зафиксирован процесс передачи сообщения и ответа на него:

1. **Пакет 35:** Клиент (`49688`) отправляет серверу (`12345`) команду или текст длиной 19 байт (`Len=19`) с флагом `PSH`.
2. **Пакет 36:** Сервер подтверждает получение этих данных, отправляя пустой пакет (`Len=0`) с флагом `ACK` и номером подтверждения `Ack=20` (исходный `Seq=1` + `19` байт данных).
3. **Пакет 37:** Сервер обрабатывает команду и отправляет ответ клиенту длиной 14 байт (`Len=14`) с флагами `[PSH, ACK]`.
4. **Пакет 38:** Клиент подтверждает получение ответа от сервера отправкой `ACK` с номером `Ack=15` (исходный `Seq=1` + `14` байт).

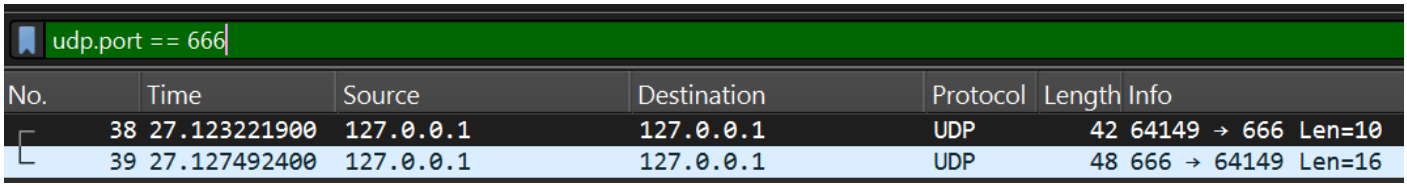


Задание 7:

```
Автоматически подключено к UDP-серверу 127.0.0.1:666

Доступные команды:
 - load <имя_файла> (запросить файл у сервера)
 - disconnect <адрес> <порт> (отключиться от сервера)
 - exit (выход из программы)

> load test2
=== Содержимое файла ===
its test message
========================
> exit
PS C:\AIPOS_II28\reports\Kanevega>
```



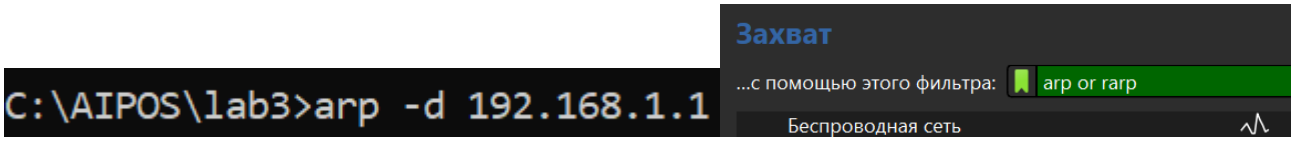
- **Source Port (Порт отправителя):** `64149`. Это динамический (эпоемерный) порт, который операционная система временно выделила программе-клиенту для отправки запроса.

- **Destination Port (Порт получателя):** `666`. Это прослушиваемый порт твоего UDP-сервера, на который было направлено сообщение.

- **Length (Длина):** `18` байт. Указывает общий размер UDP-датаграммы. Он складывается из обязательного 8-байтового заголовка самого UDP и 10 байт полезной нагрузки (UDP payload/Data), переданной твоей программой.

- **Checksum (Контрольная сумма):** `0x3caa [unverified]`. Используется для проверки того, что данные не были искажены при передаче. Пометка `[unverified]` означает, что Wireshark не производил математическую проверку этой суммы, так как перехват шел на локальном интерфейсе (`NPF_Loopback`).

Задание 8:




```
C:\AIPOS\lab3>ping 192.168.1.1

Обмен пакетами с 192.168.1.1 по с 32 байтами данных:
Ответ от 192.168.1.1: число байт=32 время=10мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=33мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=1мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=2мс TTL=64

Статистика Ping для 192.168.1.1:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 1мсек, Максимальное = 33 мсек, Среднее = 11 мсек

C:\AIPOS\lab3>arp -d 192.168.1.1

C:\AIPOS\lab3>ping 192.168.1.1

Обмен пакетами с 192.168.1.1 по с 32 байтами данных:
Ответ от 192.168.1.1: число байт=32 время=14мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=2мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=1мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=1мс TTL=64

Статистика Ping для 192.168.1.1:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 1мсек, Максимальное = 14 мсек, Среднее = 4 мсек
```

```
▸ Frame 1: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{6772C42A-5DA8-4
▸ Ethernet II, Src: InfinityWire_54:58:68 (3c:46:45:54:58:68), Dst: Intel_18:ad:cb (ac:19:8e:18:ad:cb)
▾ Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: InfinityWire_54:58:68 (3c:46:45:54:58:68)
    Sender IP address: 192.168.1.1
    Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
    Target IP address: 192.168.1.5
```

```
▸ Frame 2: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{6772C42A-5DA8-4
▸ Ethernet II, Src: Intel_18:ad:cb (ac:19:8e:18:ad:cb), Dst: InfinityWire_54:58:68 (3c:46:45:54:58:68)
▾ Address Resolution Protocol (reply)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: reply (2)
    Sender MAC address: Intel_18:ad:cb (ac:19:8e:18:ad:cb)
    Sender IP address: 192.168.1.5
    Target MAC address: InfinityWire_54:58:68 (3c:46:45:54:58:68)
    Target IP address: 192.168.1.1
```

1. Анализ ARP-запроса (Frame 1) На первом скриншоте представлен пакет запроса, инициированный роутером для поиска MAC-адреса компьютера.

- **Hardware type:** `Ethernet (1)` — указывает тип аппаратной сети.
- **Protocol type:** `IPv4 (0x0800)` — протокол сетевого уровня, для которого ищется MAC-адрес.
- **Opcode:** `request (1)` — код операции подтверждает, что это запрос на разрешение адреса.
- **Sender MAC address:** `3c:46:45:54:58:68` — физический адрес отправителя запроса (роутера InfinityWire).
- **Sender IP address:** `192.168.1.1` — логический IP-адрес роутера.
- **Target MAC address:** `00:00:00:00:00:00` — целевой MAC-адрес обнулен, так как отправитель его еще не знает и именно его пытается выяснить.
- **Target IP address:** `192.168.1.5` — IP-адрес компьютера, физический адрес которого запрашивается.

2. Анализ ARP-ответа (Frame 2) На втором скриншоте показан ответ компьютера на полученный запрос.

- **Opcode:** `reply (2)` — код операции изменился, указывая, что это ответное сообщение.
- **Sender MAC address:** `ac:19:8e:18:ad:cb` — ключевое поле ответа. Компьютер сообщает свой физический MAC-адрес.
- **Sender IP address:** `192.168.1.5` — IP-адрес компьютера.
- **Target MAC address:** `3c:46:45:54:58:68` — MAC-адрес роутера, которому направляется ответ.
- **Target IP address:** `192.168.1.1` — IP-адрес роутера.

Вывод: В ходе выполнения лабораторной работы были освоены практические приемы перехвата и детального анализа сетевого трафика с использованием программы Wireshark. Были успешно применены фильтры захвата и отображения для изоляции специфического трафика, а также собраны статистические данные (процентное соотношение протоколов, размеры пакетов, диаграммы взаимодействия).